Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 6

Тема: Асоціативні структури даних: словники та множини

Мета роботи: Опанувати практичні навички проектування та застосування асоціативних структур даних і хеш-таблиць мовою Python. Навчитися реалізовувати базові операції створення, модифікації, безпечного доступу та видалення записів у словниках (dict), застосовувати спеціалізовані методи get(), setdefault(), pop(), update(), а також будувати оптимізовані інвертовані індекси за допомогою генераторів словників (Dict Comprehensions). Опанувати частотний аналіз системних подій із використанням класу collections.Counter та навчитися виконувати теоретико-множинні операції над змінюваними (set) і незмінними (frozenset) множинами для розв'язання задач інвентаризації мережевих ресурсів, аудиту прав доступу користувачів та розрахунку метрик схожості системних конфігурацій.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп'ютерна інженерія»
Освітня програма Комп'ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та математична постановка задачі

Мета роботи: опанувати проєктування та застосування асоціативних структур даних мовою Python; виконувати CRUD-операції зі словниками (dict), використовувати методи get(), setdefault(), pop() та update(), будувати інвертовані індекси за допомогою Dict Comprehension, виконувати частотний аналіз за допомогою collections.Counter та застосовувати теоретико-множинні операції над множинами set і frozenset для аудиту прав доступу.

1.1. Індивідуальне завдання (варіант 2)

Параметр	**Значення**
Предметна область	користувачі Active Directory
Структура запису	Username → {Email, Department, Roles_Set, Is_Active}
Аудит множин	права користувача проти шаблону посади (відділу)
Частотний аналіз	Counter входів у домен

1.2. Математичний апарат

Словник і множина в CPython побудовані на хеш-таблиці. Індекс комірки визначається побітовою маскою від хеш-значення ключа, де 2^m — розмір індексної таблиці:

$$i = h(k) \;\&\; (2^m - 1)$$

Колізії розв'язуються відкритою адресацією з псевдовипадковим зондуванням, де змінна perturb спочатку дорівнює хешу й на кожному кроці зсувається праворуч на 5 біт:

$$j_{r+1} = (5 \cdot j_r + 1 + perturb_r) \bmod 2^m, \quad perturb_{r+1} = perturb_r >> 5$$

Завдяки цьому середня складність пошуку, вставки та видалення становить O(1). Для аудиту прав кожному користувачу ставиться у відповідність множина фактичних ролей A та нормативний шаблон B його відділу. Використовуються такі операції:

- надлишкові права (потенційно небезпечні): A \ B;
- відсутні обов'язкові права: B \ A;

– легітимні права: A ∩ B;

– усі враховані права: A ∪ B.

Ступінь відповідності оцінюється коефіцієнтом подібності Жаккара:

$$J(A, B) = |A \cap B| / |A \cup B| = |A \cap B| / (|A| + |B| - |A \cap B|), \quad 0 \le J \le 1$$

Значення J = 1 означає повну ідентичність фактичних прав і шаблону, J = 0 — відсутність спільних прав. Якщо обидві множини порожні, за домовленістю приймається J = 1.

2. Опис структури бази даних варіанта

Реєстр користувачів Active Directory — це словник, де ключем є унікальне ім’я користувача Username (рядок, тобто незмінний хешований тип), а значенням — вкладений словник із чотирма полями:

Поле	**Тип**	**Призначення**
email	str	службова адреса електронної пошти користувача
department	str	відділ (IT, Finance, HR, Sales); за ним визначається шаблон ролей
roles	set[str]	множина наданих ролей; множина виключає дублікати й дає перевірку членства за O(1)
is_active	bool	ознака активного облікового запису

Початковий реєстр містить 10 облікових записів: чотири відділи, вісім активних і два неактивні (nhavrylenko та vlysenko). Нормативні шаблони ролей зберігаються у словнику ROLE_TEMPLATES, значення якого — незмінні множини frozenset, щоб шаблон не можна було випадково змінити під час аудиту:

Відділ	**Шаблон ролей (Baseline)**
IT	domain_users, vpn_access, remote_desktop, admin_console, file_share_read, monitoring_view
Finance	domain_users, file_share_read, file_share_write, payroll_access, erp_access
HR	domain_users, file_share_read, hr_portal, recruiting_tool
Sales	domain_users, file_share_read, crm_access, vpn_access

Для частотного аналізу використовується журнал із 35 успішних входів (список імен користувачів). У ньому навмисно присутні дві аномалії: входи неактивного облікового запису nhavrylenko та входи облікового запису svc_backup2, якого немає в реєстрі. Запис okuzmenko активний, але жодного входу не має.

3. Алгоритмічний аналіз

3.1. Операції зі словником (network_inventory.py)

Операція	Реалізація	Складність
Створення запису	users_db[username] = info	O(1) у середньому
Оновлення полів	users_db[username].update(info)	O(m), m — кількість полів
Безпечний пошук	db.get(username, {}).get("email", "—")	O(1)
Гарантування ключа	user.setdefault("roles", set())	O(1)
Вилучення	users_db.pop(username, None)	O(1), без KeyError

Метод get() лише читає значення й повертає значення за замовчуванням для відсутнього ключа, не змінюючи словник. Метод setdefault() за відсутності ключа вставляє значення за замовчуванням і повертає його, тому функція grant_role за одну операцію гарантує наявність множини ролей і додає до неї нову роль. Метод pop(key, None) видаляє запис і повертає його, а для відсутнього ключа повертає None, замість того щоб згенерувати KeyError.

3.2. Інвертований індекс ролей

Індекс «роль → список користувачів» будується в два етапи. Спочатку множинним генератором збираються всі унікальні ролі реєстру, потім Dict Comprehension для кожної ролі формує відсортований список користувачів, у яких вона є:

```
all_roles = {role for data in users_db.values() for role in data.get("roles", set())}
return {
    role: sorted(name for name, data in users_db.items() if role in data.get("roles", set()))
    for role in sorted(all_roles)
}
```

Складність побудови становить $O(R \cdot U)$, де R — кількість унікальних ролей, U — кількість користувачів, оскільки для кожної ролі переглядається весь реєстр; для навчального обсягу даних це прийнятно. Для великих реєстрів ефективнішим є одноразовий прохід із накопиченням через setdefault(role, []).append(name) за $O(\Sigma|roles_u|)$.

3.3. Теоретико-множинний аудит (security_audit.py)

Функція audit_user_roles використовує методи difference(), intersection() та union(), які повертають нові множини й не змінюють вхідні дані. Складність різниці $A \setminus B$ становить $O(|A|)$, перетину — $O(\min(|A|, |B|))$, об'єднання — $O(|A| + |B|)$, тому аудит одного користувача виконується за час, лінійний за кількістю ролей. Статус облікового запису визначається за такими правилами:

Статус	Умова
КРИТИЧНО	обліковий запис неактивний, але зберігає хоча б одну роль (еталон — порожня множина)
НАДЛИШОК	активний користувач має ролі поза шаблоном ($A \setminus B \neq \varnothing$)
НЕДОСТАТНЬО	надлишку немає, але бракує обов'язкових ролей шаблону ($B \setminus A \neq \varnothing$)
ОК	множина ролей збігається з шаблоном, $J = 1$

Індекс Жаккара не відрізняє надлишкові права від відсутніх: він лише вимірює загальну розбіжність. З точки зору безпеки надлишкові права небезпечніші за відсутні, тому окремо виводяться обидві різниці $A \setminus B$ та $B \setminus A$.

3.4. Частотний аналіз і множинні перевірки журналу

Клас Counter підраховує входи за один прохід потоку за O(N); метод most_common(3) вибирає три найактивніші облікові записи. Додатково Counter із генератором підсумовує входи за відділами. Множинні операції над ключами журналу й реєстру виявляють три аномалії: невідомі облікові записи (logged \ known), входи неактивних записів (logged ∩ known із фільтром is_active) та активні записи без жодного входу (known \ logged).

4. Програмний код

4.1. Файл network_inventory.py

```
"""
Лабораторна робота № 6. Модуль 1. Варіант 2.
Інвентаризація користувачів Active Directory: маніпуляції словниками (dict),
безпечний доступ через get()/setdefault(), оновлення через update(),
вилучення через pop() та інвертований індекс «роль -> користувачі»
за допомогою Dict Comprehension.

Структура запису: Username -> {email, department, roles (set), is_active}
Освітній компонент: Програмування на Python.
"""

import textwrap
import time

REPORT_WIDTH: int = 88


def create_initial_user_database() -> dict[str, dict]:
    """
    Створює первинний реєстр користувачів Active Directory.
    Ключ: Username (str). Значення: словник параметрів облікового запису.
    """
    users_db = {
        "asmirnova": {
            "email": "a.smirnova@corp.local",
            "department": "IT",
            "roles": {"domain_users", "vpn_access", "remote_desktop",
                      "admin_console", "file_share_read", "monitoring_view"},
            "is_active": True,
        },
        "pkovalenko": {
            "email": "p.kovalenko@corp.local",
            "department": "IT",
            "roles": {"domain_users", "vpn_access", "remote_desktop",
"admin_console",
                      "file_share_read", "file_share_write", "db_write"},
            "is_active": True,
        },
        "nhavrylenko": {
            "email": "n.havrylenko@corp.local",
            "department": "IT",
            "roles": {"admin_console", "vpn_access"},
            "is_active": False,
        },
        "mpetrenko": {
            "email": "m.petrenko@corp.local",
            "department": "Finance",
            "roles": {"domain_users", "file_share_read", "file_share_write",
                      "payroll_access", "erp_access", "admin_console"},
```

```
            "is_active": True,
        },
        "okuzmenko": {
            "email": "o.kuzmenko@corp.local",
            "department": "Finance",
            "roles": {"domain_users", "file_share_read", "payroll_access"},
            "is_active": True,
        },
        "dshevchenko": {
            "email": "d.shevchenko@corp.local",
            "department": "HR",
            "roles": {"domain_users", "file_share_read", "hr_portal",
"recruiting_tool"},
            "is_active": True,
        },
        "ibondar": {
            "email": "i.bondar@corp.local",
            "department": "HR",
            "roles": {"domain_users", "hr_portal", "vpn_access",
"remote_desktop"},
            "is_active": True,
        },
        "tmelnyk": {
            "email": "t.melnyk@corp.local",
            "department": "Sales",
            "roles": {"domain_users", "file_share_read", "crm_access",
"vpn_access"},
            "is_active": True,
        },
        "kromanenko": {
            "email": "k.romanenko@corp.local",
            "department": "Sales",
            "roles": {"domain_users", "crm_access"},
            "is_active": True,
        },
        "vlysenko": {
            "email": "v.lysenko@corp.local",
            "department": "Sales",
            "roles": {"domain_users", "crm_access", "vpn_access"},
            "is_active": False,
        },
    }
    return users_db


def add_or_update_user(users_db: dict, username: str, user_info: dict) -> None:
    """
    Додає нового користувача або оновлює поля існуючого облікового запису
    за допомогою методу update() (злиття словників).
    """
    if username in users_db:
        users_db[username].update(user_info)
    else:
        users_db[username] = user_info
```

```python
def remove_user(users_db: dict, username: str) -> dict | None:
    """
    Вилучає обліковий запис методом pop() із значенням за замовчуванням None,
    що запобігає виникненню KeyError для неіснуючого користувача.
    """
    removed = users_db.pop(username, None)
    if removed is None:
        print(f"Попередження: спроба видалення неіснуючого користувача
'{username}'.")
    return removed


def get_user_email(users_db: dict, username: str) -> str:
    """Безпечний пошук e-mail через get(): для відсутнього ключа повертає '—'
замість KeyError."""
    return users_db.get(username, {}).get("email", "—")


def grant_role(users_db: dict, username: str, role: str) -> None:
    """
    Надає користувачеві роль. Метод setdefault() гарантує наявність множини
ролей:
    якщо ключа 'roles' ще немає, він створюється порожньою множиною.
    """
    users_db[username].setdefault("roles", set()).add(role)


def revoke_role(users_db: dict, username: str, role: str) -> None:
    """Відкликає роль; discard() не генерує виняток, якщо ролі в множині
немає."""
    users_db[username].setdefault("roles", set()).discard(role)


def build_inverted_role_index(users_db: dict) -> dict[str, list[str]]:
    """
    Будує інвертований індекс «роль -> відсортований список користувачів»
    за допомогою Dict Comprehension.
    """
    # 1. Множина всіх унікальних ролей у реєстрі (set comprehension)
    all_roles = {role for data in users_db.values() for role in data.get("roles",
set())}

    # 2. Генерація індексу Role -> [Usernames]
    return {
        role: sorted(name for name, data in users_db.items() if role in
data.get("roles", set()))
        for role in sorted(all_roles)
    }


def print_inventory_report(users_db: dict, inverted_index: dict) -> None:
```

```
    """Виводить відформатований звіт інвентаризації та інвертований індекс
ролей."""
    header = (f"| {'Username':<16} | {'E-mail':<32} | {'Відділ':<10} | "
              f"{'Активний':^8} | {'Ролей':^6} |")
    print("=" * REPORT_WIDTH)
    print(f"{'РЕЄСТР КОРИСТУВАЧІВ ACTIVE DIRECTORY':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    print(header)
    print("-" * REPORT_WIDTH)
    for username, data in sorted(users_db.items()):
        active = "Так" if data["is_active"] else "Ні"
        print(f"| {username:<16} | {data['email']:<32} | {data['department']:<10}
| "
              f"{active:^8} | {len(data['roles']):^6d} |")

    print("=" * REPORT_WIDTH)
    print(f"{'ІНВЕРТОВАНИЙ ІНДЕКС: РОЛЬ -> СПИСОК
КОРИСТУВАЧІВ':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    for role, names in inverted_index.items():
        line = f"  {role:<18}: " + ", ".join(names)
        # Довгі списки переносяться з відступом, щоб не виходити за ширину звіту
        print(textwrap.fill(line, width=REPORT_WIDTH, subsequent_indent=" " *
22))
    print("=" * REPORT_WIDTH + "\n")


def main() -> None:
    """Головна точка входу модуля інвентаризації користувачів."""
    t_start = time.perf_counter()

    # 1. Створення реєстру
    users_db = create_initial_user_database()
    print(f"Початковий розмір реєстру: {len(users_db)} облікових записів")

    # 2. CRUD: додавання нового користувача
    add_or_update_user(users_db, "ysavchenko", {
        "email": "y.savchenko@corp.local",
        "department": "HR",
        "roles": {"domain_users"},
        "is_active": True,
    })

    # 3. CRUD: оновлення існуючого запису через update() (зміна e-mail)
    add_or_update_user(users_db, "okuzmenko", {"email":
"olena.kuzmenko@corp.local"})

    # 4. Безпечний доступ get(): існуючий та відсутній користувач
    print(f"get_user_email('okuzmenko') -> {get_user_email(users_db,
'okuzmenko')}")
    print(f"get_user_email('ghost_user') -> {get_user_email(users_db,
'ghost_user')}")

    # 5. setdefault(): надання ролі та відкликання зайвого права
```

```
    grant_role(users_db, "ysavchenko", "hr_portal")
    revoke_role(users_db, "pkovalenko", "db_write")

    # 6. CRUD: вилучення звільненого користувача та неіснуючого запису
    removed = remove_user(users_db, "vlysenko")
    print(f"Вилучено запис: vlysenko (відділ {removed['department']})")
    remove_user(users_db, "ghost_user")
    print()

    # 7. Інвертований індекс і звіт
    role_index = build_inverted_role_index(users_db)
    print_inventory_report(users_db, role_index)

    elapsed = time.perf_counter() - t_start
    print(f"Час виконання модуля інвентаризації: {elapsed * 1000:.3f} мс\n")


if __name__ == "__main__":
    main()
```

4.2. Файл security_audit.py

```
"""
Лабораторна робота № 6. Модуль 2. Варіант 2.
Аудит прав користувачів Active Directory проти шаблону відділу (множини),
розрахунок коефіцієнта Жаккара та частотний аналіз входів засобами
collections.Counter.
Освітній компонент: Програмування на Python.
"""

import time
from collections import Counter

from network_inventory import create_initial_user_database

REPORT_WIDTH: int = 88

# Заголовки стовпців із зворотним слешем винесено за межі f-рядків (Python < 3.12
не допускає \ у виразах)
COL_EXCESS: str = "|A\\B|"
COL_MISSING: str = "|B\\A|"

# Нормативні шаблони ролей за відділом (Baseline). frozenset - незмінні множини
ROLE_TEMPLATES: dict[str, frozenset[str]] = {
    "IT": frozenset({"domain_users", "vpn_access", "remote_desktop",
                     "admin_console", "file_share_read", "monitoring_view"}),
    "Finance": frozenset({"domain_users", "file_share_read", "file_share_write",
                          "payroll_access", "erp_access"}),
    "HR": frozenset({"domain_users", "file_share_read", "hr_portal",
"recruiting_tool"}),
```

```
    "Sales": frozenset({"domain_users", "file_share_read", "crm_access",
"vpn_access"}),
}

# Потік успішних входів у домен (журнал автентифікації)
LOGIN_EVENTS: list[str] = [
        "kromanenko", "tmelnyk", "mpetrenko", "svc_backup2",
        "asmirnova", "ibondar", "svc_backup2", "mpetrenko",
        "asmirnova", "pkovalenko", "ibondar", "pkovalenko",
        "dshevchenko", "mpetrenko", "pkovalenko", "dshevchenko",
        "asmirnova", "pkovalenko", "asmirnova", "dshevchenko",
        "asmirnova", "nhavrylenko", "pkovalenko", "pkovalenko",
        "tmelnyk", "tmelnyk", "pkovalenko", "mpetrenko",
        "tmelnyk", "asmirnova", "asmirnova", "asmirnova",
        "mpetrenko", "nhavrylenko", "asmirnova",
]


def audit_user_roles(actual_roles: set[str], baseline_roles: frozenset[str]) ->
dict:
    """
    Теоретико-множинний аудит ролей користувача відносно шаблону:
    надлишкові права A \\ B, відсутні обов'язкові права B \\ A, перетин A ∩ B
    та коефіцієнт Жаккара J(A, B) = |A ∩ B| / |A ∪ B|.
    """
    excess = actual_roles.difference(baseline_roles)
    missing = baseline_roles.difference(actual_roles)
    compliant = actual_roles.intersection(baseline_roles)
    union = actual_roles.union(baseline_roles)

    jaccard = len(compliant) / len(union) if union else 1.0

    return {
        "compliant": compliant,
        "excess": excess,
        "missing": missing,
        "jaccard": jaccard,
    }


def classify_status(user: dict, result: dict) -> str:
    """Визначає статус обліковки: КРИТИЧНО / НАДЛИШОК / НЕДОСТАТНЬО / ОК."""
    if not user["is_active"] and user["roles"]:
        return "КРИТИЧНО"          # неактивний обліковий запис зберігає права
доступу
    if result["excess"]:
        return "НАДЛИШОК"
    if result["missing"]:
        return "НЕДОСТАТНЬО"
    return "ОК"


def audit_all_users(users_db: dict) -> dict[str, dict]:
    """
```

```
    Виконує аудит усіх користувачів. Для неактивних записів еталоном є порожня
    множина прав (порожній frozenset): будь-яка збережена роль є порушенням.
    """
    results = {}
    for username, user in users_db.items():
        baseline = ROLE_TEMPLATES[user["department"]] if user["is_active"] else
frozenset()
        res = audit_user_roles(user["roles"], baseline)
        res["status"] = classify_status(user, res)
        res["baseline_size"] = len(baseline)
        results[username] = res
    return results


def analyze_login_frequency(events: list[str]) -> Counter:
    """Частотний аналіз потоку входів за допомогою collections.Counter."""
    return Counter(events)


def print_audit_table(users_db: dict, results: dict[str, dict]) -> None:
    """Виводить зведену таблицю аудиту та деталі порушень."""
    header = (f"| {'Користувач':<15} | {'Відділ':<8} | {'Статус':^12} | {'|
A|':^4} | "
              f"{'|B|':^4} | {COL_EXCESS:^6} | {COL_MISSING:^6} | {'J(A,B)':^8}
|")
    print("=" * REPORT_WIDTH)
    print(f"{'АУДИТ ПРАВ КОРИСТУВАЧІВ ПРОТИ ШАБЛОНУ ВІДДІЛУ':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    print(header)
    print("-" * REPORT_WIDTH)
    for username in sorted(results):
        r, u = results[username], users_db[username]
        print(f"| {username:<15} | {u['department']:<8} | {r['status']:^12} | "
              f"{len(u['roles']):^4d} | {r['baseline_size']:^4d} |
{len(r['excess']):^6d} | "
              f"{len(r['missing']):^6d} | {r['jaccard']:^8.4f} |")
    print("-" * REPORT_WIDTH)

    print(f"{'ДЕТАЛІ ПОРУШЕНЬ':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    for username in sorted(results):
        r = results[username]
        if r["status"] == "OK":
            continue
        print(f"  {username} [{r['status']}]")
        if r["excess"]:
            print(f"      Надлишкові права (A \\ B): {',
'.join(sorted(r['excess']))}")
        if r["missing"]:
            print(f"      Відсутні права   (B \\ A): {',
'.join(sorted(r['missing']))}")
    print("-" * REPORT_WIDTH)
```

```python
def print_login_analysis(users_db: dict, events: list[str]) -> None:
    """Виводить частотний аналіз входів та пов'язані множинні перевірки."""
    freq = analyze_login_frequency(events)
    total = len(events)

    print(f"{'ЧАСТОТНИЙ АНАЛІЗ ВХОДІВ (TOP-3)':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    print(f"| {'Ранг':^8} | {'Користувач':^26} | {'Кількість входів':^21} |
{'Частка від усіх':^20} |")
    print("-" * REPORT_WIDTH)
    for rank, (name, count) in enumerate(freq.most_common(3), start=1):
        share = f"{100.0 * count / total:.2f}%"
        print(f"| {rank:^8d} | {name:^26} | {count:^21d} | {share:^20} |")
    print("-" * REPORT_WIDTH)

    # Входи за відділами: Counter із генератором
    by_department = Counter(users_db[u]["department"] for u in events if u in
users_db)
    dept_line = ", ".join(f"{d}: {n}" for d, n in by_department.most_common())
    print(f"  Входи за відділами:                    {dept_line}")

    # Множинні перевірки журналу входів проти реєстру
    known, logged = set(users_db), set(freq)
    unknown_accounts = logged - known
    inactive_logins = {u for u in logged & known if not users_db[u]["is_active"]}
    dormant_active = {u for u in known - logged if users_db[u]["is_active"]}

    print(f"  Невідомі облікові записи у журналі:    {sorted(unknown_accounts) or
'-'}")
    print(f"  Входи неактивних облікових записів:    {sorted(inactive_logins) or
'-'}")
    print(f"  Активні записи без жодного входу:      {sorted(dormant_active) or
'-'}")
    print("=" * REPORT_WIDTH)


def print_summary(users_db: dict, results: dict[str, dict]) -> None:
    """Виводить підсумок аудиту."""
    active = [n for n, u in users_db.items() if u["is_active"]]
    avg_j = sum(results[n]["jaccard"] for n in active) / len(active)
    excess_total = sum(len(r["excess"]) for n, r in results.items() if
users_db[n]["is_active"])
    statuses = Counter(r["status"] for r in results.values())

    print(f"\n{'ПІДСУМОК АУДИТУ':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    print(f"  Облікових записів / активних:          {len(users_db)} /
{len(active)}")
    print(f"  Середній J(A, B) активних записів:     {avg_j:.4f} ({avg_j *
100:.2f}%)")
    print(f"  Сумарно надлишкових прав у активних:   {excess_total}")
    print(f"  Розподіл статусів:                     {dict(statuses)}")
    print("=" * REPORT_WIDTH + "\n")
```

```python
def main() -> None:
    """Головна точка входу модуля аудиту доступу."""
    t_start = time.perf_counter()

    users_db = create_initial_user_database()
    results = audit_all_users(users_db)

    print()
    print_audit_table(users_db, results)
    print_login_analysis(users_db, LOGIN_EVENTS)
    print_summary(users_db, results)

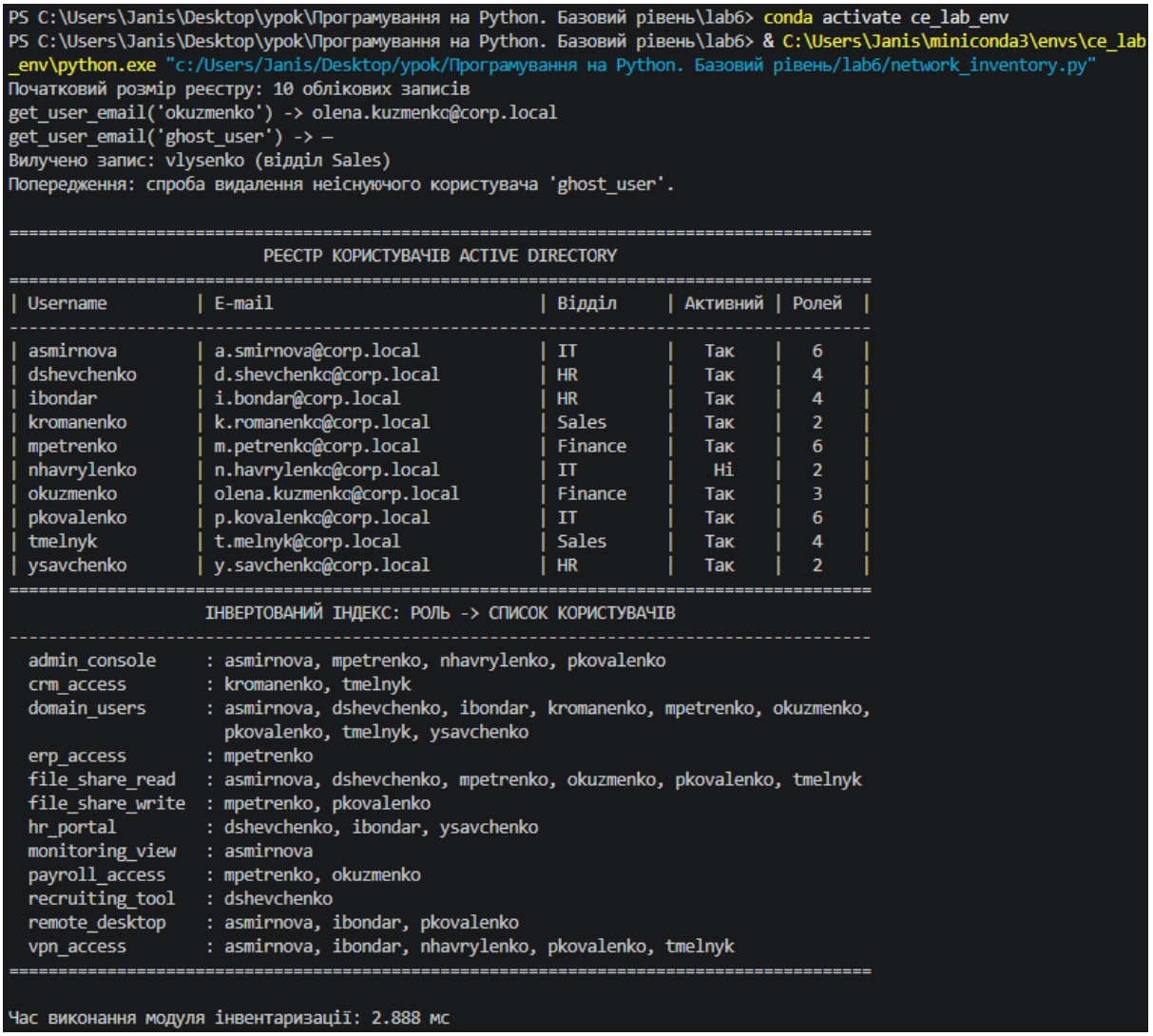
    elapsed = time.perf_counter() - t_start
    print(f"Час виконання модуля аудиту доступу: {elapsed * 1000:.3f} мс\n")


if __name__ == "__main__":
    main()
```

5. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командами python network_inventory.py та python security_audit.py.

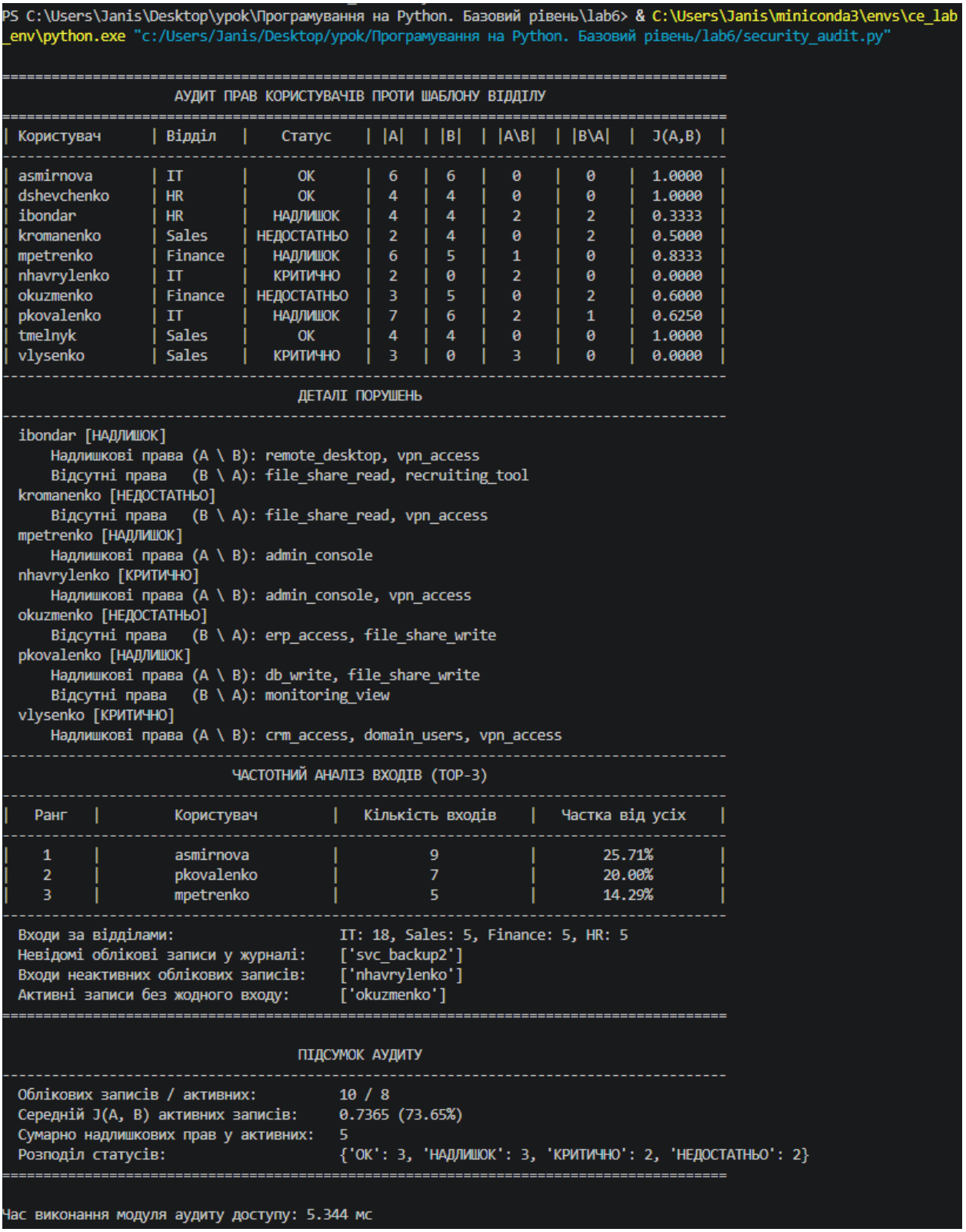
5.1. Результат роботи network_inventory.py



Скріншот 1 — Реєстр користувачів та інвертований індекс ролей

Реєстр змінено CRUD-операціями: додано ysavchenko (HR), оновлено e-mail користувача okuzmenko через update(), вилучено vlysenko через pop(). Виклик get_user_email для відсутнього користувача повертає «—» без винятку, а повторне вилучення неіснуючого ghost_user дає лише попередження. Роль hr_portal надано користувачу ysavchenko через setdefault(), а роль db_write відкликано в pkovalenko, тому в індексі її вже немає. Підсумковий розмір реєстру — 10 записів (10 + 1 − 1). Інвертований індекс показує, що роль domain_users мають дев’ять користувачів, а admin_console — чотири, зокрема неактивний nhavrylenko.

5.2. Результат роботи security_audit.py



Скріншот 2 — Аудит прав проти шаблонів відділів та частотний аналіз входів

Аудит виконано для початкового реєстру з 10 записів. Перевірка вручну: для pkovalenko A містить 7 ролей, шаблон IT — 6; перетин складається з 5 ролей, об’єднання — з 7 + 6 − 5 = 8, тому J = 5/8 = 0,625. Для ibondar $|A \cap B| = 2$, $|A \cup B| = 4 + 4 - 2 = 6$, J = 2/6 = 0,3333. Для mpetrenko надлишкова роль

admin_console дає J = 5/6 = 0,8333. Середній J активних записів: (1 + 1 + 0,3333 + 0,5 + 0,8333 + 0,6 + 0,625 + 1) / 8 = 0,7365.

Два неактивні облікові записи (nhavrylenko та vlysenko) зберігають ролі, зокрема admin_console та vpn_access, тому отримали статус КРИТИЧНО: після звільнення чи блокування права слід відкликати. Counter визначив лідером за входами asmirnova (9 із 35, тобто 25,71 %), далі pkovalenko (7, 20,00 %) і mpetrenko (5, 14,29 %). Журнал виявив невідомий запис svc_backup2 (2 входи), вхід неактивного nhavrylenko (2 входи) та активний запис okuzmenko без жодного входу — усі три випадки потребують розслідування. Сумарно в активних користувачів 5 надлишкових прав: ibondar — 2, pkovalenko — 2, mpetrenko — 1.

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи: https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab6

7. Висновки

У ході лабораторної роботи реалізовано інвентаризацію та аудит користувачів Active Directory для варіанта 2 на основі хеш-таблиць Python. Вкладений словник «Username → параметри» дав доступ до запису за O(1), а методи get(), setdefault(), pop() та update() забезпечили безпечне читання, створення, оновлення й вилучення записів без виняткових ситуацій для відсутніх ключів.

Dict Comprehension дозволив побудувати інвертований індекс «роль → користувачі» одним виразом і швидко побачити, хто має конкретне право (наприклад, admin_console). Для великих реєстрів ефективнішою є побудова за один прохід із setdefault(), оскільки індекс на основі генератора має складність O(R · U).

Теоретико-множинні операції та індекс Жаккара дали кількісну оцінку відповідності прав шаблону відділу: середній J активних користувачів становить 0,7365, виявлено 5 надлишкових прав і 2 неактивні облікові записи, що зберігають права, зокрема права адміністрування. Використання frozenset для шаблонів гарантує, що еталон не зміниться під час аудиту. Частотний аналіз засобами Counter за один прохід визначив найактивніших користувачів і разом із множинними перевірками виявив невідомий обліковий запис та входи заблокованого користувача. Це показує практичну роль множин та лічильників у задачах аудиту кібербезпеки.

Контрольні питання:

1. Як влаштована компактна хеш-таблиця (Compact Dict) в інтерпретаторі CPython? Яким чином розділення на розріджену таблицю indices та щільний масив entries гарантує збереження порядку вставки елементів?

Словник складається з двох масивів. Розріджена таблиця indices містить короткі цілі числа (1, 2, 4 або 8 байтів залежно від розміру), кожне з яких є або порожнім маркером, або номером запису. Щільний масив entries зберігає трійки (хеш, ключ, значення), які додаються строго послідовно в порядку вставки. Пошук обчислює i = hash & mask, читає idx = indices[i] і звертається до entries[idx]; при колізії зондування продовжується. Оскільки нові записи завжди дописуються в кінець entries, а ітерація обходить цей масив, порядок вставки зберігається. Видалений запис позначається порожнім, а при перебудові таблиці масив ущільнюється. Така схема заощаджує пам’ять порівняно зі старою розрідженою таблицею, де кожна комірка зберігала всю трійку.

2. Сформулюйте контракт хешованості об'єкта (hashability). Чому об'єкти типу set та list не можуть бути елементами множини або ключами словника, а об'єкти типу frozenset та tuple — можуть?

Хешований об’єкт має метод __hash__, значення якого не змінюється протягом життя об’єкта, і метод __eq__, узгоджений з ним: якщо a == b, то hash(a) == hash(b). Змінювані контейнери list, set та dict мають __hash__ = None, бо зміна вмісту змінила б хеш, і запис «загубився» б у таблиці. frozenset незмінний, його хеш обчислюється з елементів, тому він хешований. Кортеж хешований, лише якщо хешовані всі його елементи: кортеж зі списком усередині викличе TypeError.

3. Чим семантично відрізняється робота методу dict.get(key, default) від методу dict.setdefault(key, default)? У яких інженерних алгоритмах застосування setdefault() є найбільш доцільним?

get() лише повертає значення або default і не змінює словник. setdefault() повертає наявне значення, а якщо ключа немає, вставляє default у словник і повертає його, тобто мутує словник. Він доцільний для групування й накопичення: d.setdefault(key, []).append(item) створює список за потреби й одразу додає до нього елемент. У цій роботі так працює grant_role: user.setdefault("roles", set()).add(role). Слід пам’ятати, що аргумент default обчислюється при кожному виклику, навіть коли ключ уже є, тому для великих обсягів зручніший collections.defaultdict.

4. Розкрийте математичний та інженерний зміст коефіцієнта подібності Жаккара (*Jaccard Similarity*). Як за допомогою операцій над множинами intersection та union розрахувати ступінь відповідності конфігурації еталону?

Коефіцієнт Жаккара J(A, B) = |A ∩ B| / |A ∪ B| вимірює частку спільних елементів серед усіх елементів двох множин; він змінюється від 0 до 1. Для аудиту A — фактична конфігурація, B — еталон: у коді J = len(a.intersection(b)) / len(a.union(b)). Наприклад, для pkovalenko перетин містить 5 ролей, об’єднання — 8, тобто J = 0,625. Величина 1 − J слугує відстанню між

конфігураціями. Недолік метрики в тому, що вона не розрізняє надлишкові й відсутні права, тому разом із нею виводять A \ B та B \ A.

5. Опишіть алгоритмічні переваги класу collections.Counter над звичайним словником dict при підрахунку частоти подій у високонавантажених потоках логів.

Counter — підклас dict, призначений для підрахунку. Конструктор Counter(iterable) підраховує елементи за один прохід у вбудованому коді на C, швидше за ручний цикл d[k] = d.get(k, 0) + 1. Звернення до відсутнього елемента повертає 0 і не створює ключ, тому KeyError не виникає. Метод most_common(k) вибирає k найчастіших елементів за O(n log k). Крім того, Counter підтримує арифметику (+, −, &, |), update() та subtract() і працює як мультимножина.

6. Які методи множин мутують об'єкт на місці (наприклад, difference_update()), а які повертають новий екземпляр множини? Як це впливає на використання оперативної пам'яті?

На місці працюють update(), intersection_update(), difference_update(), symmetric_difference_update(), add(), remove(), discard(), pop(), clear() та оператори |=, &=, −=, ^=. Нову множину повертають union(), intersection(), difference(), symmetric_difference(), copy() та оператори |, &, −, ^. Мутуючі методи не виділяють пам’ять під нову множину, що економить пам’ять для великих даних, але змінюють оригінал, і ця зміна видима всім посиланням на нього. Немутуючі методи безпечні для спільних об’єктів, тому в роботі аудит використовує саме їх: шаблони ROLE_TEMPLATES — це frozenset, які взагалі не мають мутуючих методів, а початкові ролі користувача потрібні для звіту.